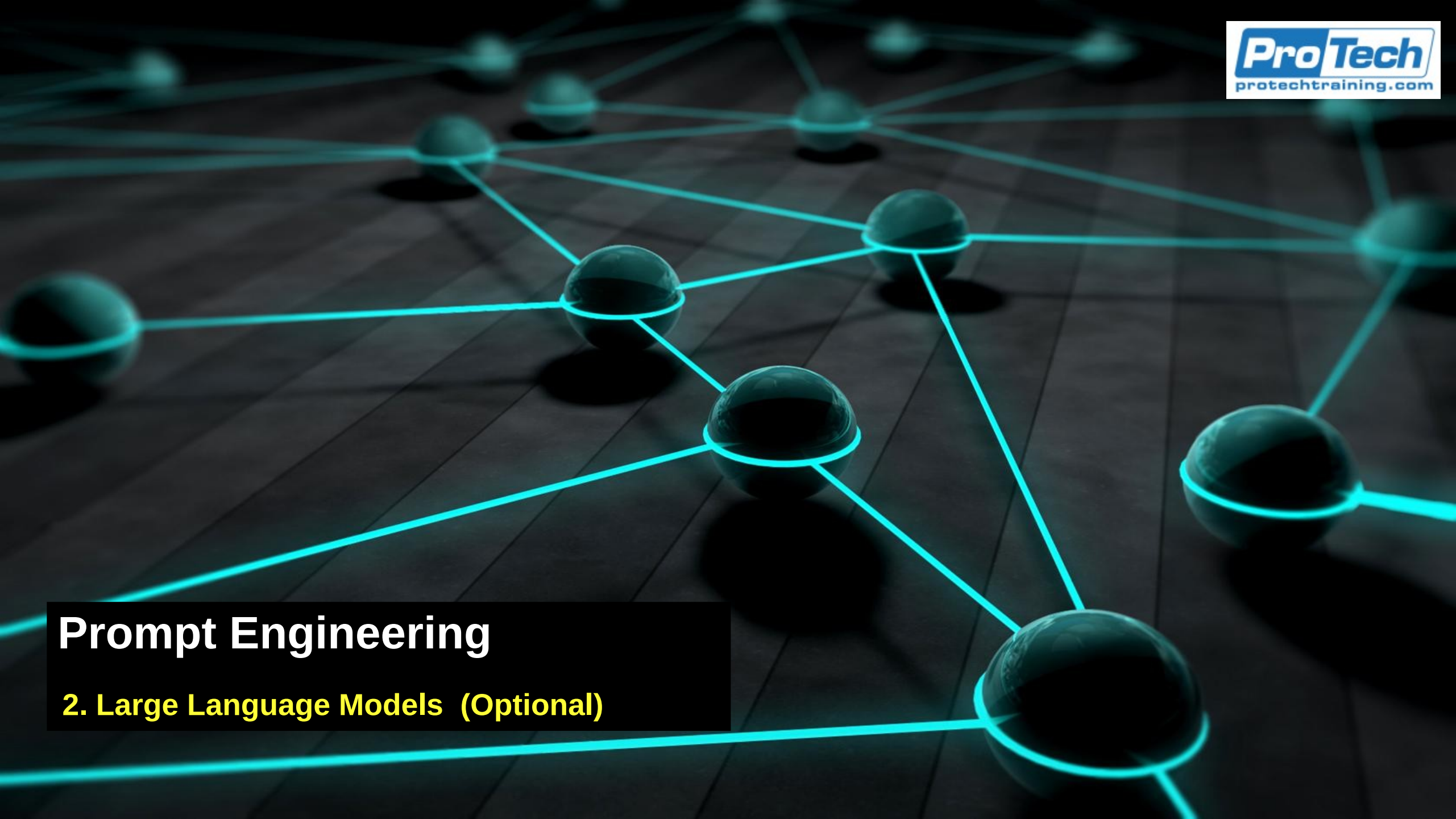


Prompt Engineering

2. Large Language Models (Optional)

Text Processing

- This includes code
- Machine learning algorithms use linear algebra, statistical methods, and other mathematical techniques to train models
  - Non-numeric training data has to be converted into a numerical form
  - Language data is a string of characters
    - Tokenization converts the string of characters into a string of words
    - Vectorization converts tokens into numerical vectors called parameters

Tokenization

- Tokenization is the process of splitting text into smaller units called tokens
  - Tokens can be
    - Words
    - Subwords
    - Characters
    - Special symbols
  - Example
    - Text : “Transformers are powerful.”
    - Tokenized: ["Transformers", "are", "powerful", "."]

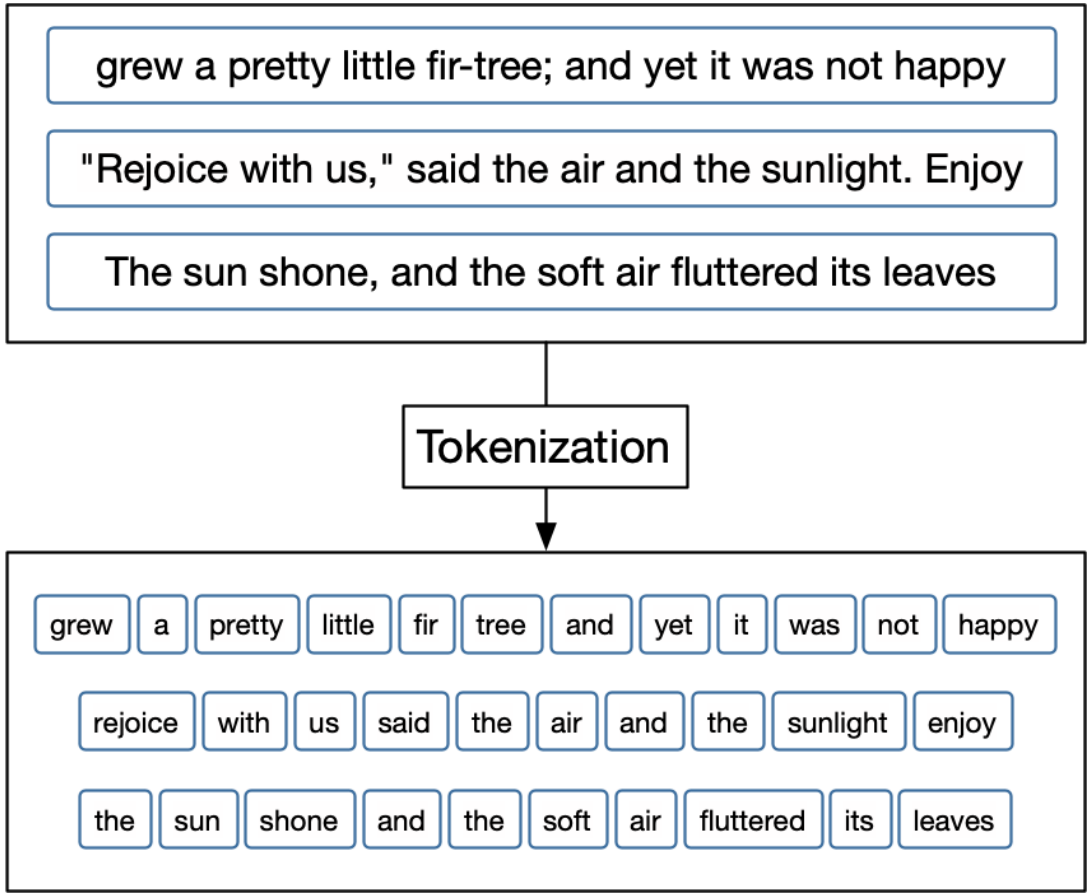


Image Credit: https://vamosarema.com/

Vector Embedding

- Because LLMs work on numbers, the tokens need to be converted into vectors
- Color terms example
  - Can be turned into vectors with three dimensions (red, green, blue) with values from 0 to 255
  - We can then determine similarity of colors by using vector distances
  - Orange (255,165,0) is closer to red (255,0,0) then blue (0,0,255)

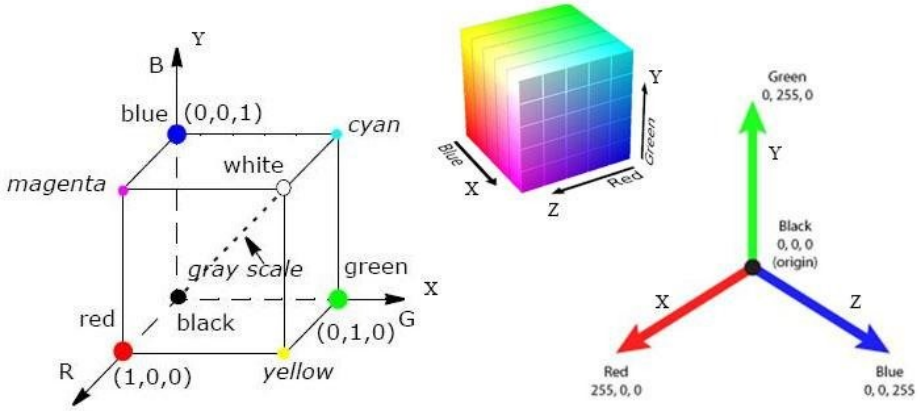


Image Credit: https://www.researchgate.net/figure/Color-distribution-of-three-dimensional-RGB-vector-space-in-a-cubical-format_fig2_349475027

Vector Embedding

- Color terms are arbitrary
  - There are different ways of vectorizing colors, most use three dimensions
  - Instead of red/green/blue, we could use magenta/yellow/cyan
- In this case, the dimension or parameters are motivated by the way we see color

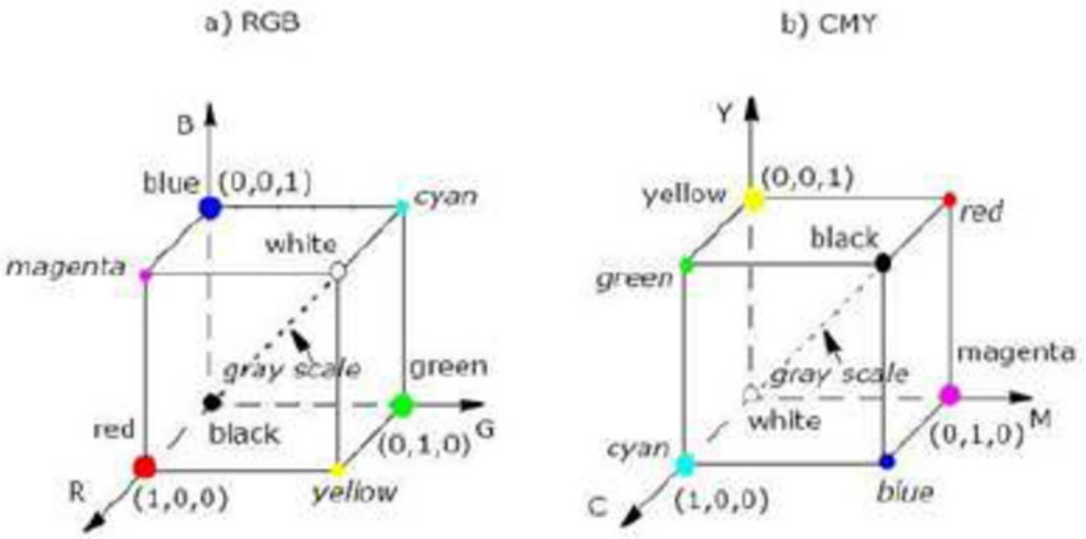


Image Credit: https://www.researchgate.net/figure/Color-distribution-of-three-dimensional-RGB-vector-space-in-a-cubical-format_fig2_349475027

Vector Embedding

- We could do something similar for other words by choosing a set of meaningful dimensions or parameters
- The selection of parameters is usually pre-selected in classical machine learning
  - Usually those that should be most useful in building the model
  - For example, income is more useful for predicting credit worthiness and shoe size is less useful

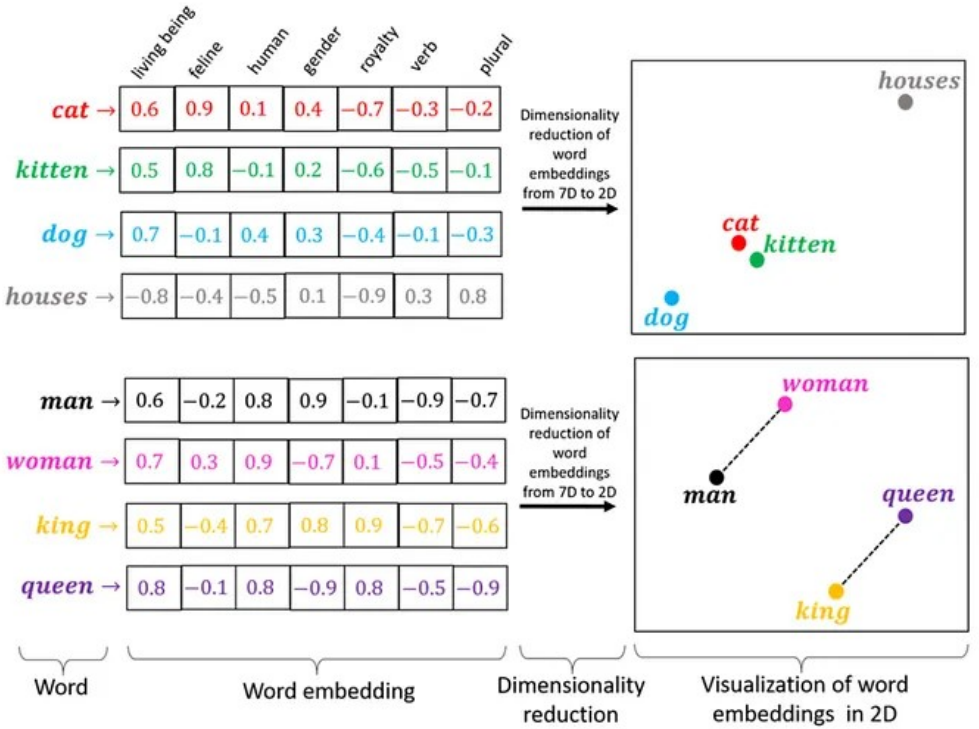
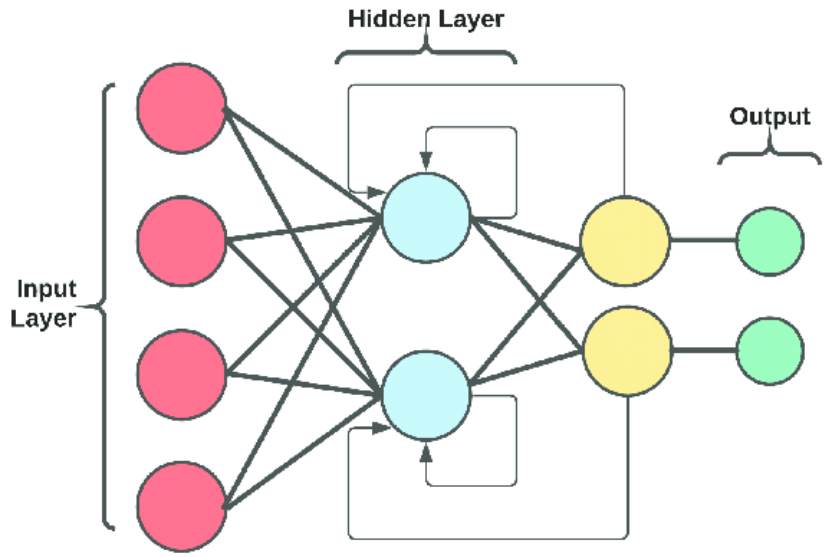


Image Credit: https://ai.plainenglish.io/heres-why-you-need-to-learn-linear-algebra-for-data-science-ad8b917a5abe

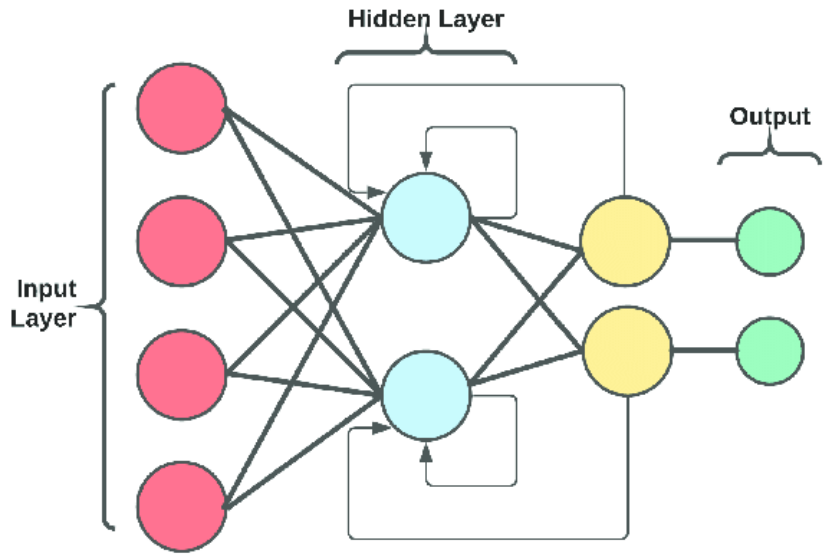
Choosing Features

- Problem, choosing the right features
- Solution: Use a neural net
  - It will do automatic feature extraction based on patterns in the data
  - But the choice of features is opaque to us
  - The more features we use, the better the performance
  - Most LLMs use billions or trillions of features



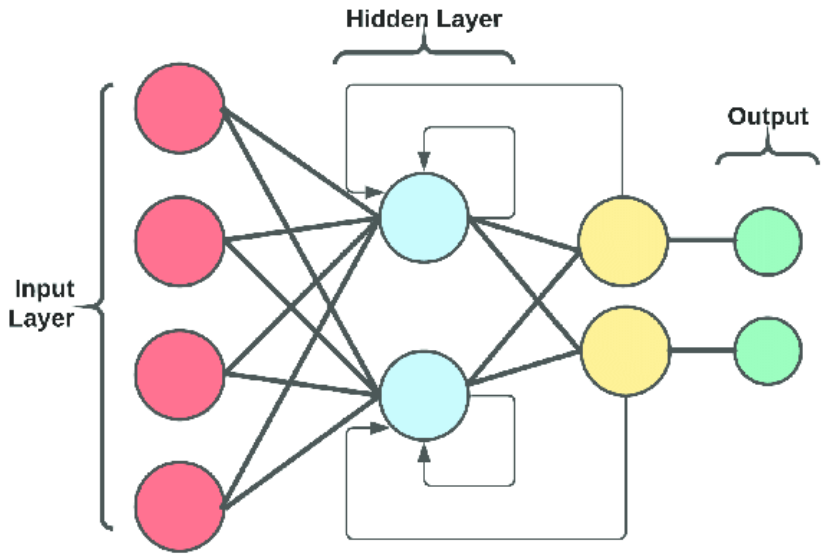
Processing Text

- Processing text is a fundamentally different problem than processing images
  - Images have fixed dimensions (224×224 pixels)
  - Text varies in length from short phrases to long documents
- Neural networks work best with
  - Fixed-size inputs
  - Independent feature vectors
- Text is
  - Ordered
  - Context-dependent
  - Potentially very long



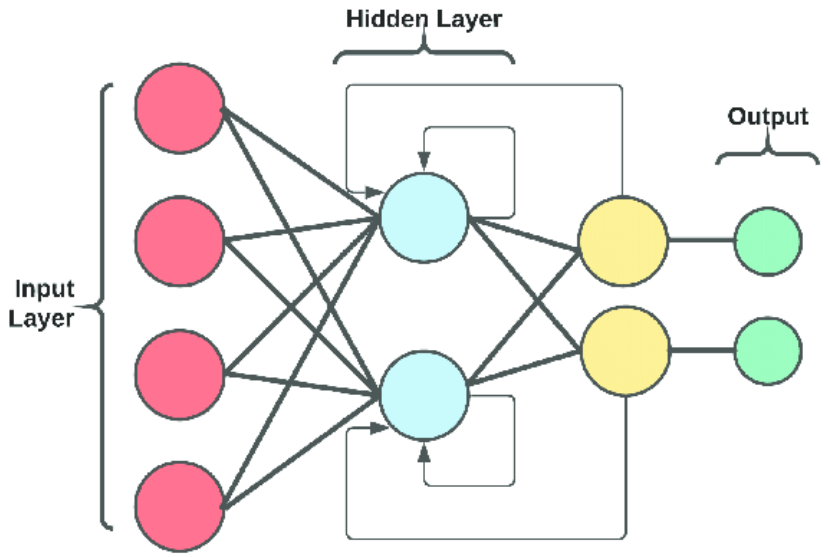
Processing Text (or Code)

- Language meaning often depends on distant words
- Example
  - “The book on the table near the window that we bought yesterday is mine.”
  - The word “is” relates to “book,” not “window”
- Early models struggled to
  - Remember information from far back in the sequence
  - Maintain context across long spans
  - This is called the long-range dependency problem



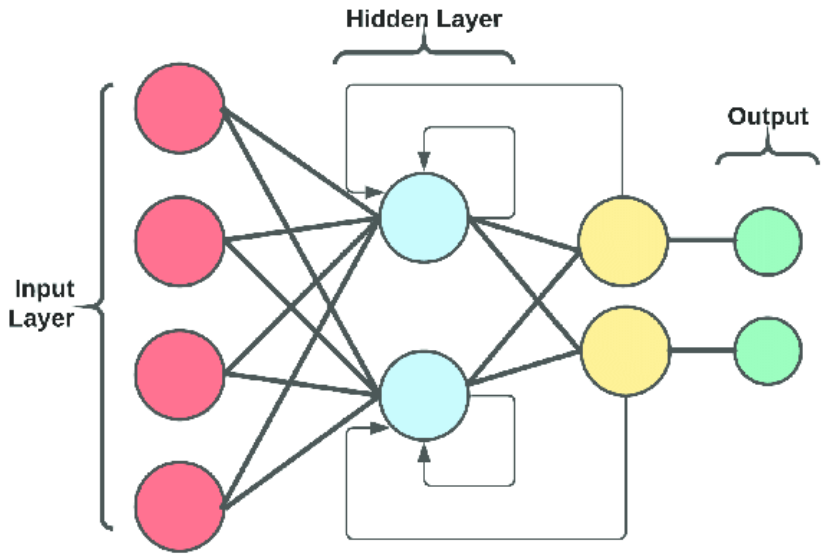
Processing Text (or Code)

- Text is symbolic
  - Words are discrete tokens
  - No natural numeric representation
  - Vocabulary can be extremely large
- Neural networks require continuous numerical input
  - That means that text must first be converted into vector embeddings
  - Early vector embeddings were limited in capturing context if there were too few dimensions



Processing Text (or Code)

- Words change meaning based on context
  - “Bank” (river vs financial)
  - “Apple” (fruit vs company)
- Early models
  - Used static word embeddings
    - e.g., Word2Vec, GloVe
  - Used the same vector for a word regardless of context
  - There was no dynamic contextual understanding



Transformers

- Transformers, instead of processing one word at a time, look at all the words at once
- This enabled
  - Full parallelization
  - Better long-range reasoning
  - Massive scalability
  - Better performance on large datasets
- Transformers were the enabling technology for LLMs

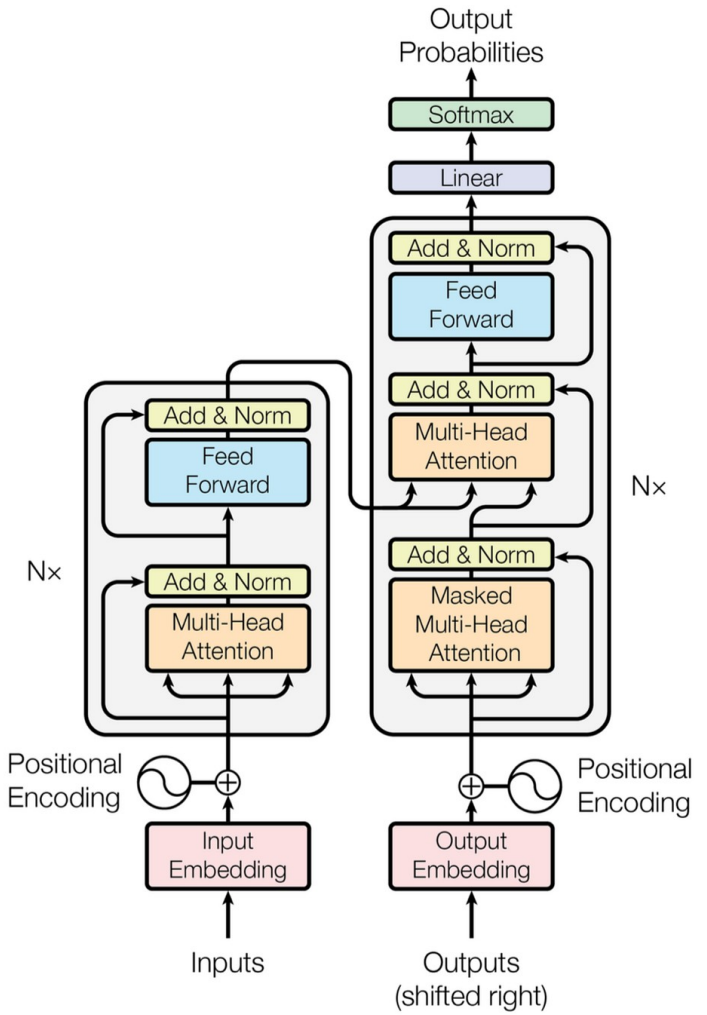


Figure 1: The Transformer - model architecture.

Transformer Architectures

- Encoder Only
- BERT
  - Bidirectional Encoder Representations from Transformers
  - Transformer-based model designed for language understanding, not text generation
- Introduced by Google in 2018 and marked a major breakthrough in NLP
- Produces contextual embeddings that can be fine-tuned for tasks like classification, question answering, and search.

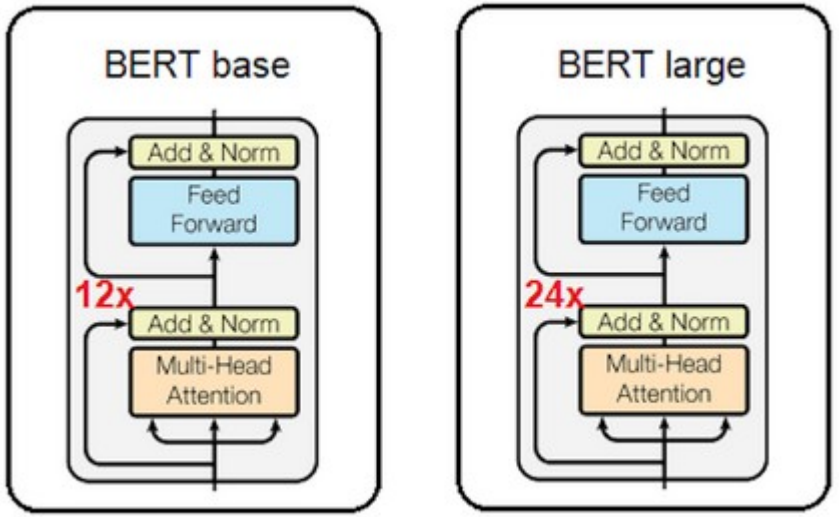


Image Credit: https://sushant-kumar.com/blog/bert

Transformer Architectures

- Decoder Only
- GPT
  - Generative Pre-trained Transformer
  - Decoder-only transformers are the architecture behind
    - GPT models
    - LLaMA
    - Claude
    - Most modern Large Language Models
  - Optimized for text generation
  - Predict the next token given all previous tokens
  - They generate text one token at a time
  - Either language or program code

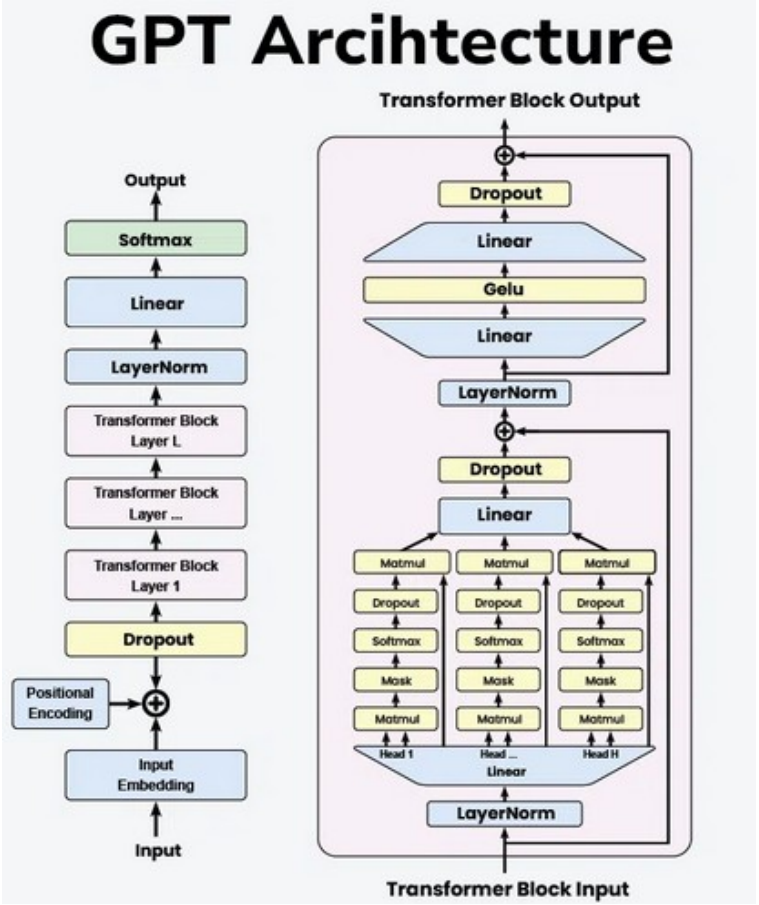


Image Credit: https://www.geeksforgeeks.org/artificial-intelligence/introduction-to-generative-pre-trained-transformer-gpt/

Transformer Architectures

- At inference time
  - Input prompt is tokenized
  - Model processes full prompt
  - Outputs probability distribution over vocabulary
  - One token is selected
  - Token is appended
  - Repeat
- This creates fluent, coherent text

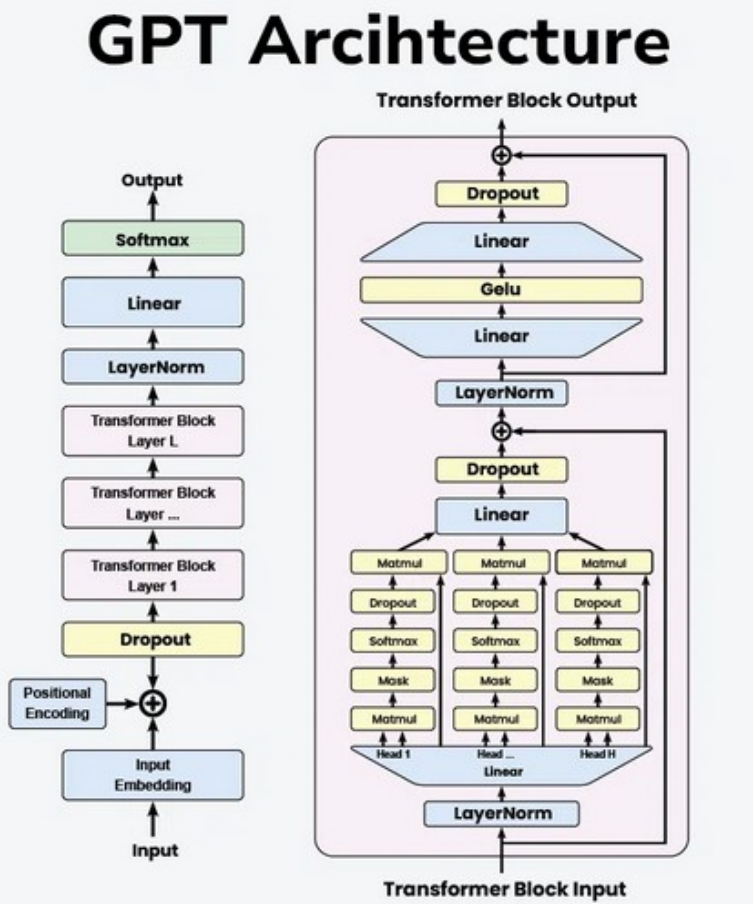


Image Credit: https://www.geeksforgeeks.org/artificial-intelligence/introduction-to-generative-pre-trained-transformer-gpt/

Transformer Architectures

- Decoder-only models work well for
  - Text generation
  - Code generation
  - Creative writing
  - Dialogue
  - Reasoning
  - In-context learning
- They can simulate
  - Classification
  - Translation
  - Summarization
  - Question answering
- All via prompting

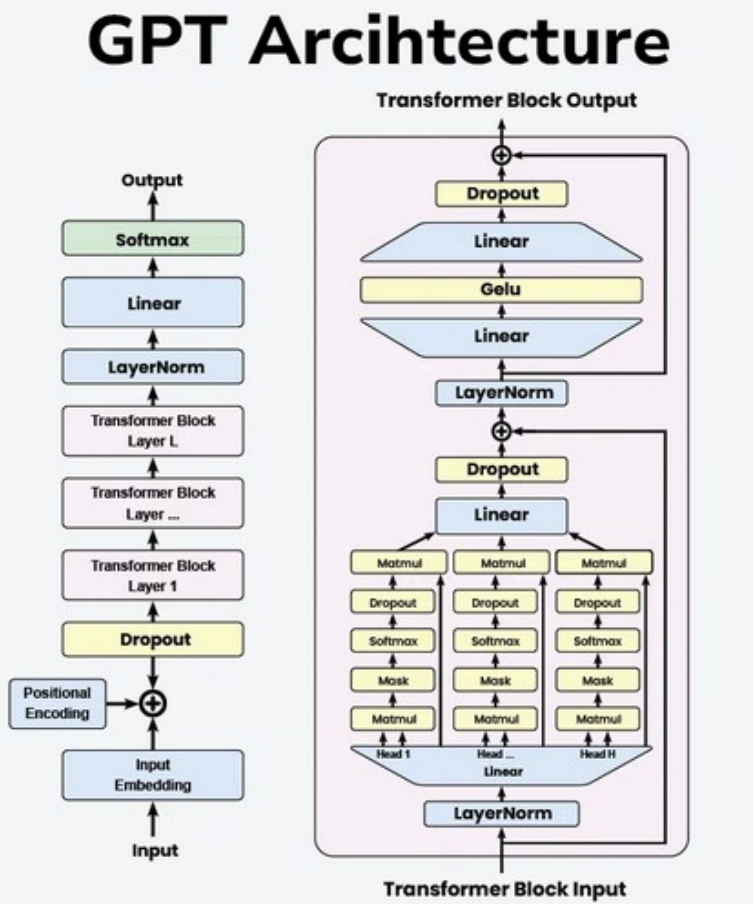
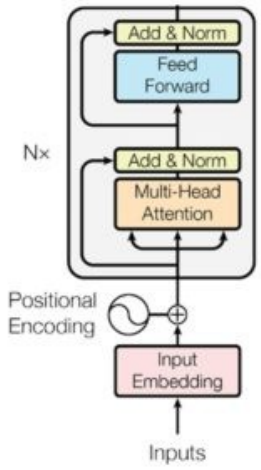


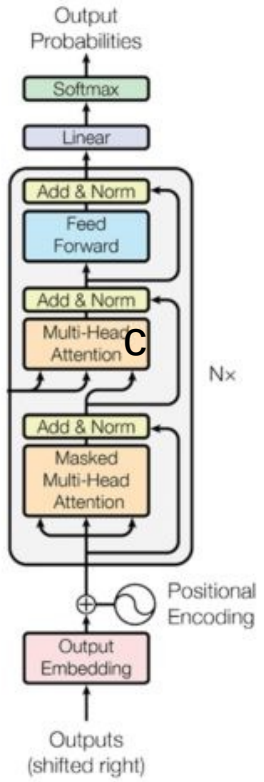
Image Credit: https://www.geeksforgeeks.org/artificial-intelligence/introduction-to-generative-pre-trained-transformer-gpt/

Transformer Architectures

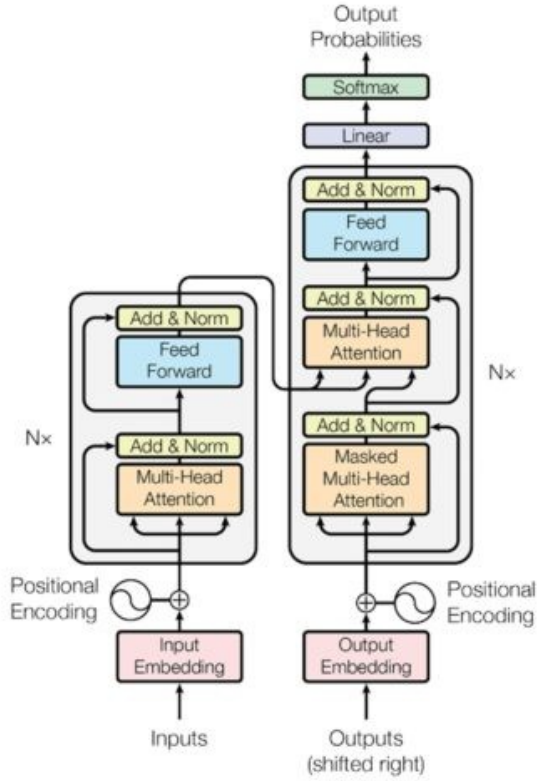
Encoder-only



Decoder-only



Encoder-Decoder



Training LLMs

- Pretraining
  - The model learns general language understanding from a vast corpus of text data
  - This often uses unsupervised learning techniques
  - Produces a base of foundation model
- Fine-tuning
  - Model is further trained on specific tasks or domains, adapting its knowledge to particular applications
    - For example, Claude code and copilot are trained on code
- Objective function
  - During training, the model aims to minimize a loss function, typically the difference between its predictions and the actual text

Image Credit: https://workhub.ai/how-llms-work/

Training LLMs

- First and largest phase
  - Where the model learns
    - Grammar
    - Syntax
    - General world knowledge
    - Patterns of reasoning
    - Statistical structure of language
- Pre-training is usually done on
  - Massive internet-scale text corpora
  - Books, articles, code, websites, etc.
  - Trillions of tokens

Image Credit: https://workhub.ai/how-llms-work/

Training LLMs

- First and largest phase
  - Where the model learns
    - Grammar
    - Syntax
    - General world knowledge
    - Patterns of reasoning
    - Statistical structure of language
- Pre-training is usually done on
  - Massive internet-scale text corpora
  - Books, articles, code, websites, etc.
  - Trillions of tokens

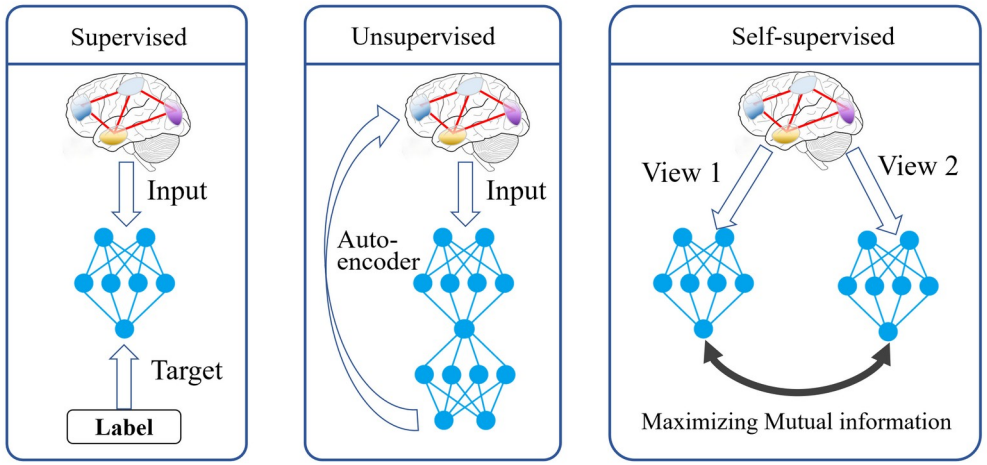


Image Credit: https://larryuestc.github.io/gate.github.io/

Training LLMs

- Most LLMs are trained using next-token prediction
  - Given previous tokens, predict the next token
  - Example input
    - “The capital of France is”
  - The model predicts:
    - “Paris”
- Called self-supervised learning
  - The training right answer comes from the data itself
  - No human labels are needed

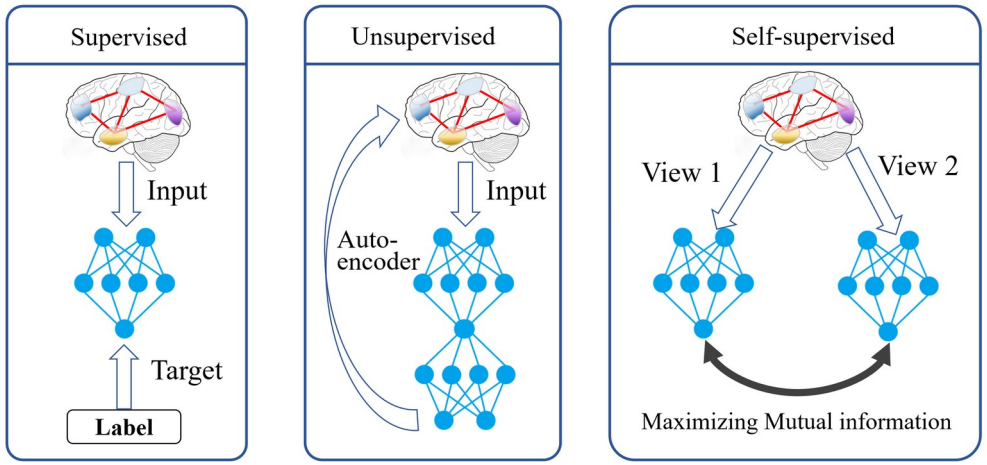


Image Credit: https://larryuestc.github.io/gate.github.io/

Training LLMs

- Although simple, this objective forces the model to learn
  - Language structure
  - Factual knowledge
  - Commonsense reasoning
  - Code patterns
  - Even some reasoning chains
- To predict the next word well, the model must understand context

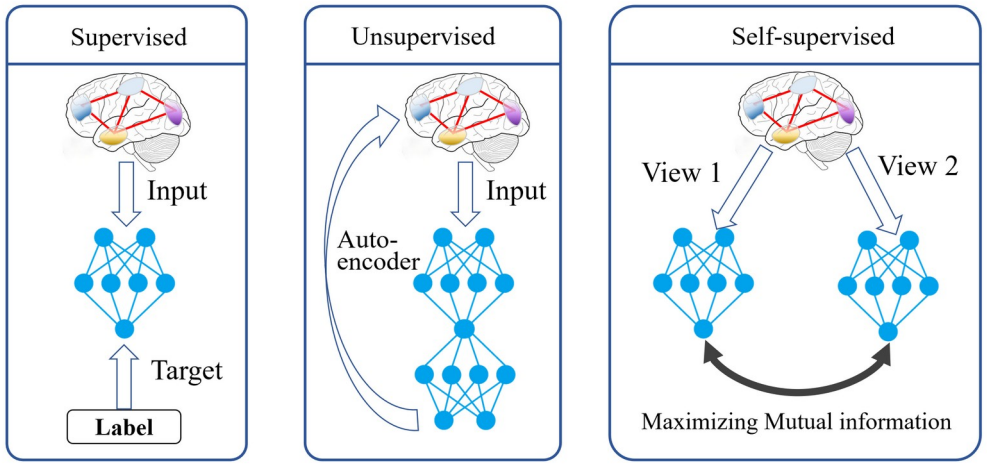


Image Credit: https://larryuestc.github.io/gate.github.io/

Base Model

- A base model is
  - Fully trained on massive text data
  - Not fine-tuned to follow instructions
  - Not optimized for safety or alignment
  - Not designed specifically for chat interactions
- It is optimized to model the statistical patterns of language
- When trained on large code bases
  - It learns statistical patterns that occur in code

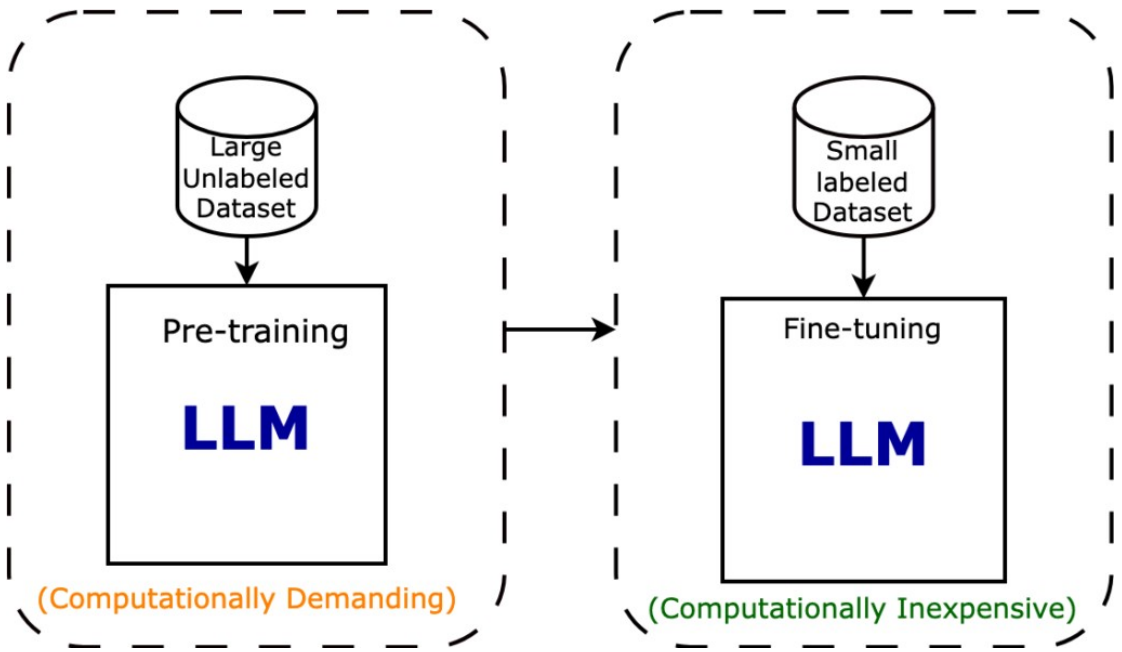


Image Credit: https://intuitivetutorial.com/2023/06/18/large-language-models-in-deep-learning/

Base Model

- A base model is is not “chat ready”
- Prompt: “Explain quantum mechanics”
  - A base model might
    - Continue like a textbook
    - Change tone mid-way
    - Provide incomplete structure
    - Produce unsafe or unfiltered content
- It has no alignment, which is
  - Shaping an AI system’s behavior so that its outputs are consistent with human values, intentions, and safety expectations
  - *This directly relates to the AI Disasters you looked at in lab 1-3*

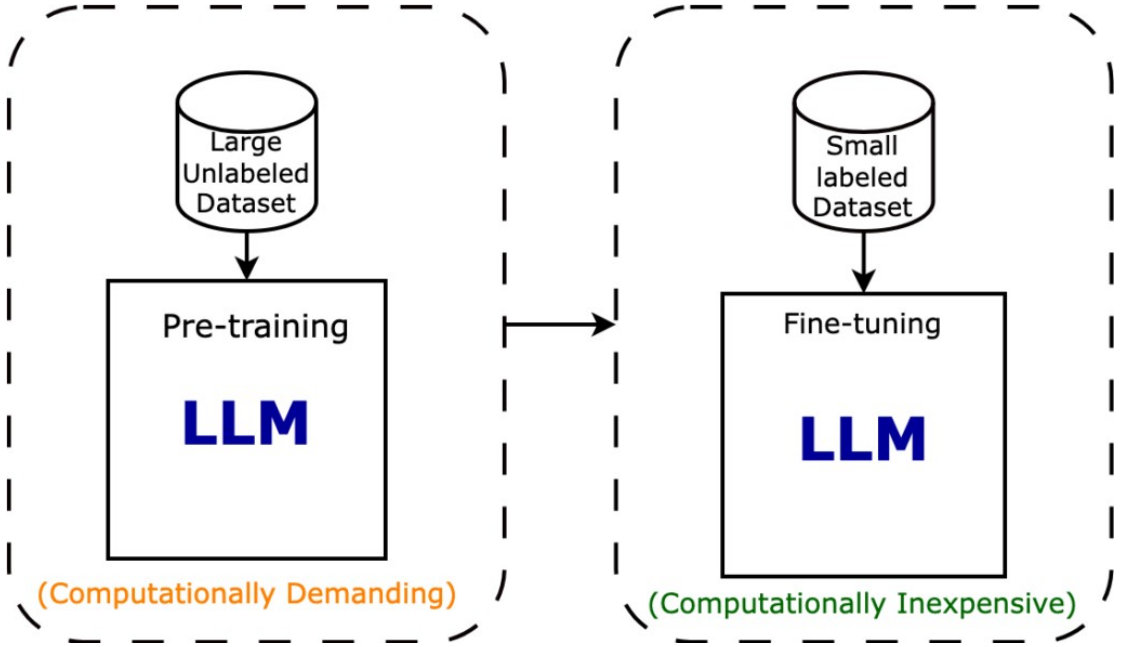


Image Credit: https://intuitivetutorial.com/2023/06/18/large-language-models-in-deep-learning/

Supervised Fine-Tuning

- SFT trains the model on curated prompt–response pairs
- The model learns to
  - Respond directly to instructions
  - Format answers cleanly
  - Follow conversational patterns
- After SFT, the model
  - Follows instructions more reliably
  - Adopts assistant-style tone
  - Structures answers clearly

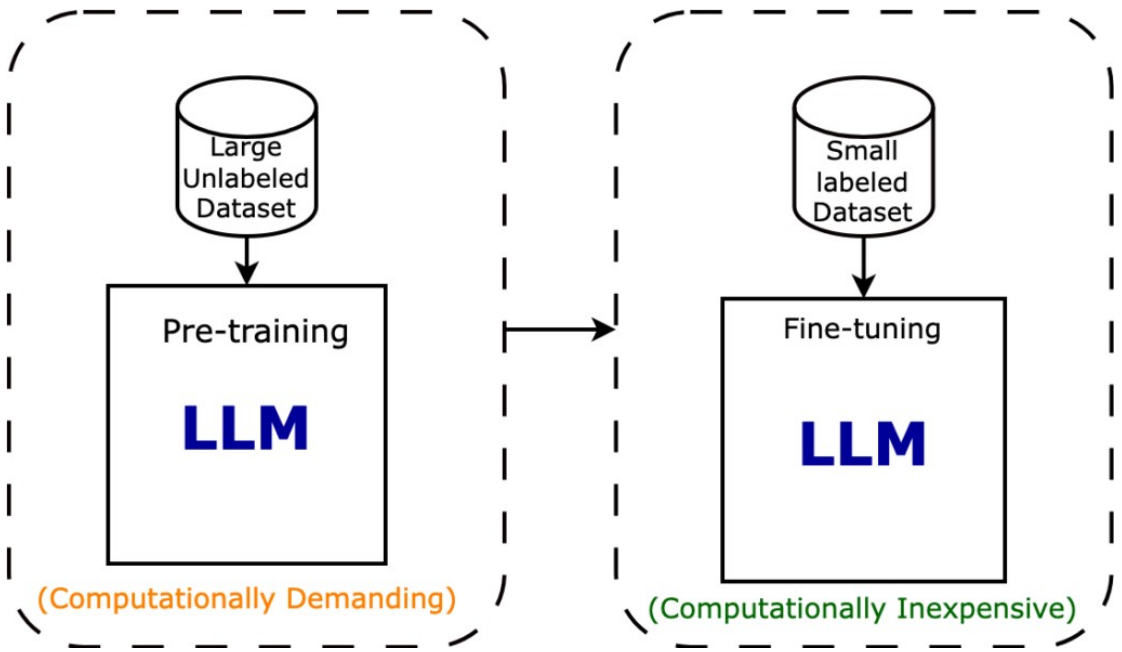


Image Credit: https://intuitivetutorial.com/2023/06/18/large-language-models-in-deep-learning/

Instruction Datasets

- Relies on carefully designed training data that teaches the model how to behave like a helpful assistant
- These datasets typically contain
  - A user request (question, task, or instruction)
  - A high-quality example response from an assistant
- The examples cover many types of tasks, such as
  - Explaining concepts
  - Solving reasoning problems
  - Writing or editing text
  - Generating or debugging code
  - Summarizing documents
  - Engaging in dialogue

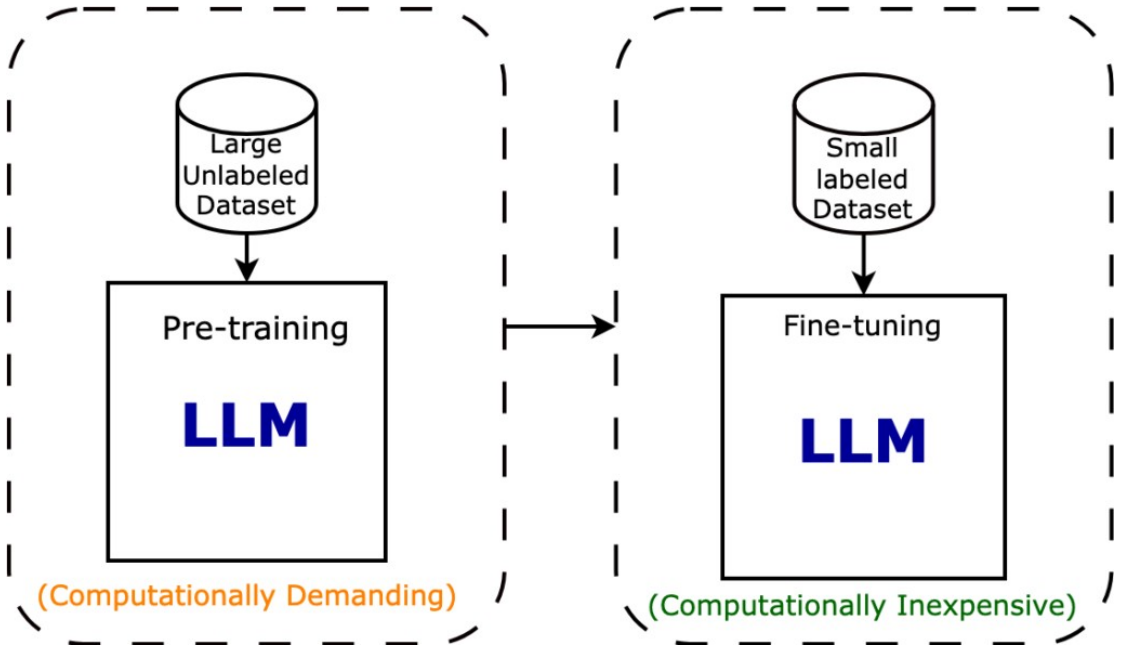


Image Credit: https://intuitivetutorial.com/2023/06/18/large-language-models-in-deep-learning/

Instruction Datasets

- Instruction datasets are created using
  - Human annotators who write ideal responses
  - Synthetic data generated by stronger or earlier models
  - Publicly available instruction-following datasets
- Good instruction data
  - Covers a wide variety of tasks and domains
  - Demonstrates clear, structured reasoning
  - Includes examples of safe refusals when needed
  - Shows appropriate tone (helpful, polite, professional)
  - Models concise and well-formatted answers
- The more diverse and well-designed the instruction data is, the better the model generalizes to new tasks

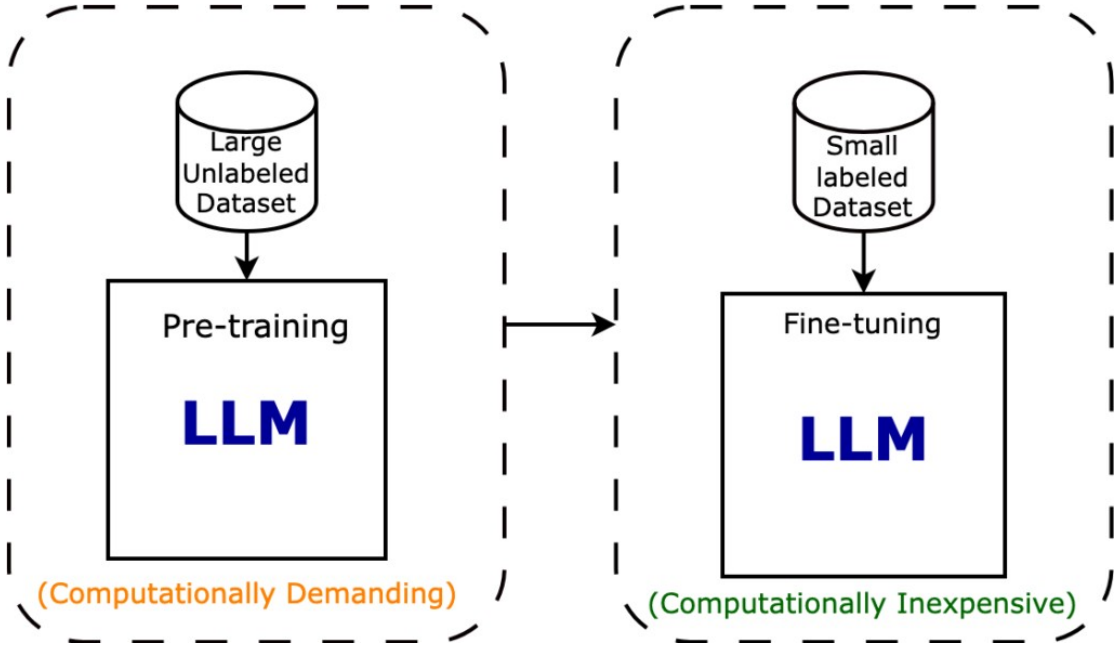


Image Credit: https://intuitivetutorial.com/2023/06/18/large-language-models-in-deep-learning/

RLHF

- Reinforcement Learning from Human Feedback (RLHF)
  - Used to align LLMs with human preferences
  - Incorporates human judgments into the training process
  - Three stages
    - The model generates multiple responses to a prompt
    - Humans rank the responses from best to worst
    - The model is adjusted to produce responses that humans prefer
- Optimizes production of outputs that are aligned with user expectations

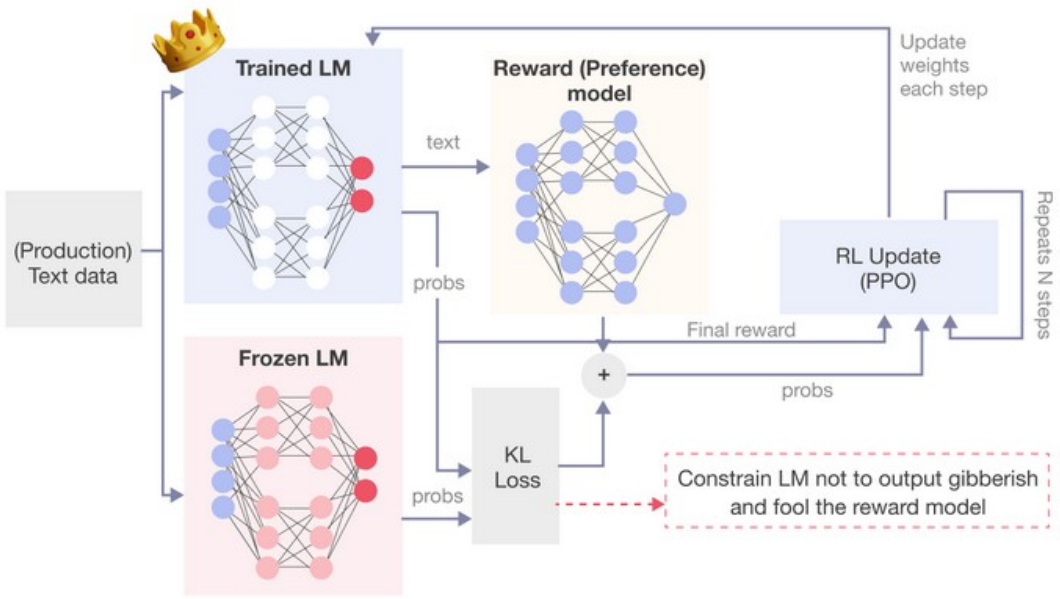


Image Credit: https://www.simform.com/blog/reinforcement-learning-from-human-feedback/

Policy

- RFHL is reinforcement learning
- In reinforcement learning, a policy is
  - A function that decides what action to take given a situation
- In the context of language models
  - The situation = the prompt
  - The action = the next token the model chooses
  - The language model itself is the policy
- RLHF adjusts the policy so that
  - Helpful responses become more likely
  - Harmful responses become less likely

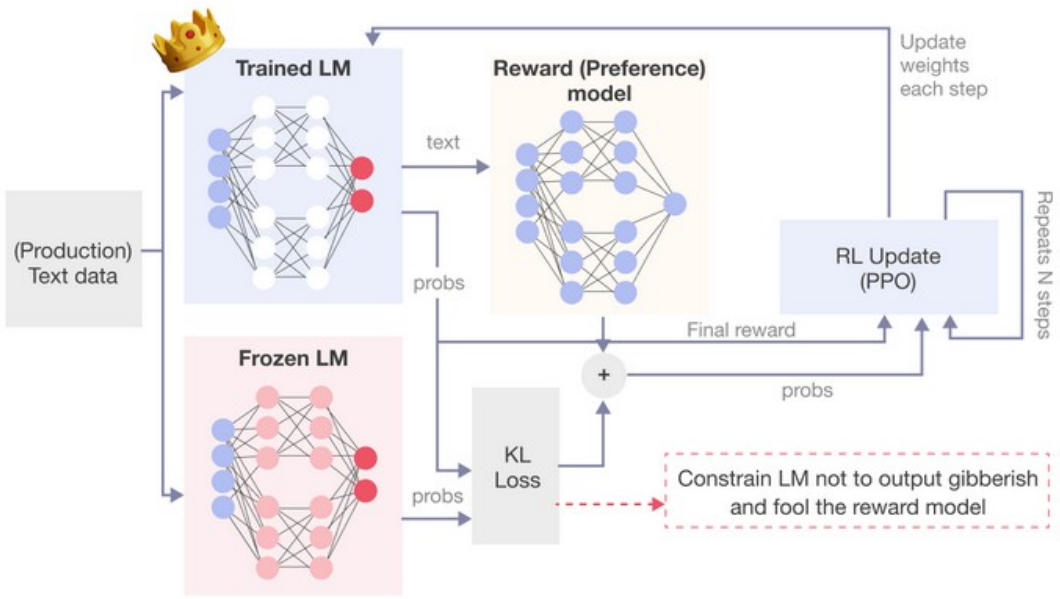


Image Credit: https://www.simform.com/blog/reinforcement-learning-from-human-feedback/

Proximal Policy Optimization

- PPO = Proximal Policy Optimization is a reinforcement learning algorithm used to
  - Carefully update the model so it improves reward without changing too drastically
- In RLHF
  - The model generates a response
  - The reward model gives it a score based on human feedback
  - PPO adjusts the model to increase probability of high-reward outputs

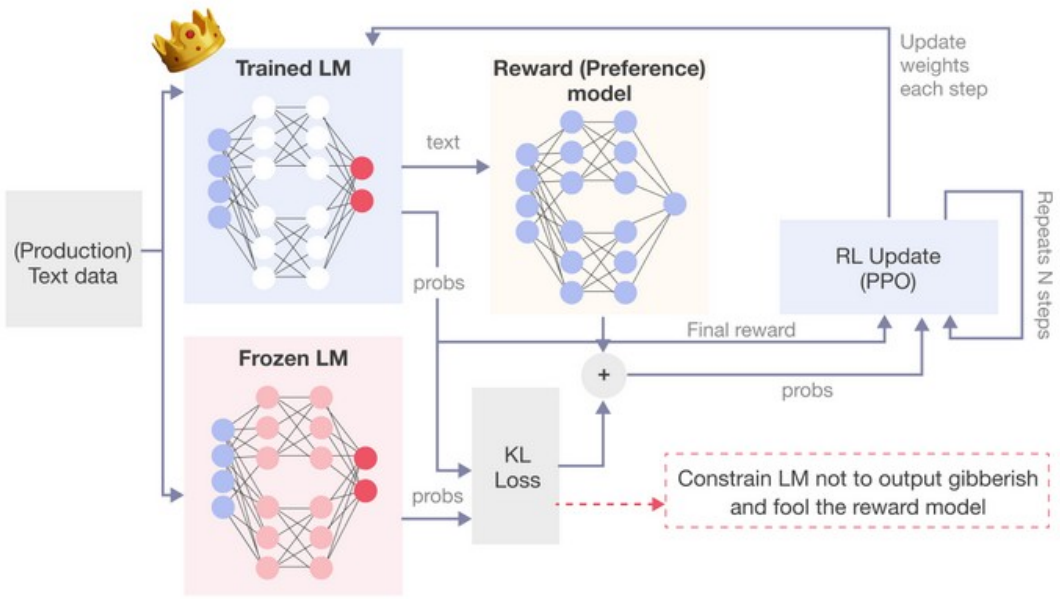


Image Credit: https://www.simform.com/blog/reinforcement-learning-from-human-feedback/

Proximal Policy Optimization

- If we modify the model only to maximize reward, problems can occur
  - The model may overfit to the reward model.
  - It may exploit weaknesses in the reward function
  - It may drift far from natural language
  - It may become unstable
- This is called reward hacking

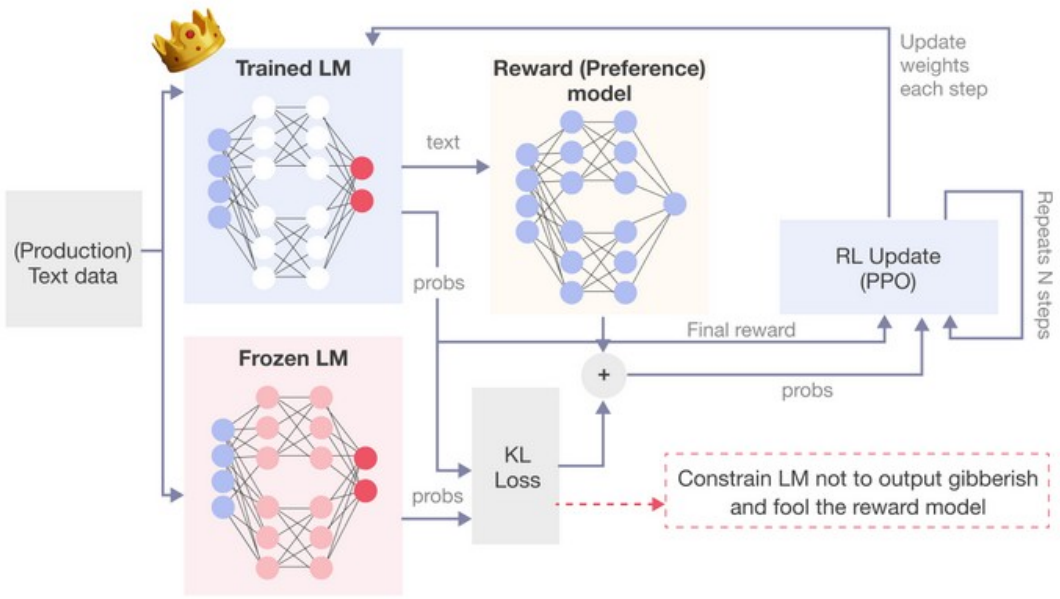


Image Credit: https://www.simform.com/blog/reinforcement-learning-from-human-feedback/

Proximal Policy Optimization

- PPO
  - Updates the model gradually and with constraints to prevent large sudden shifts
  - Ensures that LFHR improves behavior, but doesn't change the model too much at once
- PPO includes a KL penalty
  - This keeps the new policy close to the original pretrained model
  - The penalty is higher the more the answer diverges from the existing model
  - This serves to
    - Maximize reward
    - But stay close to original model distribution

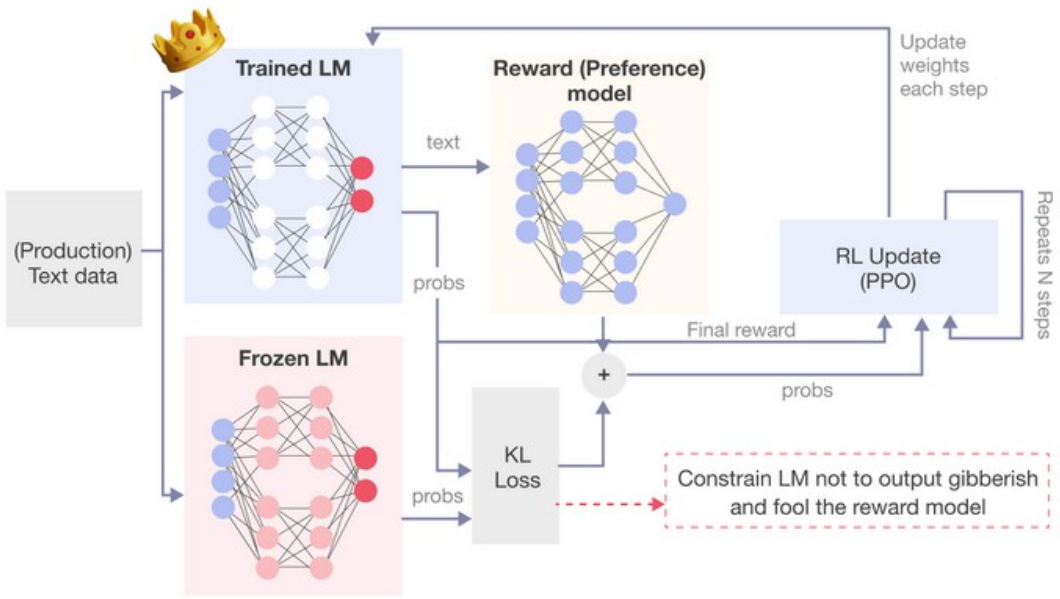


Image Credit: https://www.simform.com/blog/reinforcement-learning-from-human-feedback/

Direct Preference Optimization

- RLHF involves
  - Training a reward model from human rankings
  - Using reinforcement learning policies to optimize the model
  - Adding KL penalties to prevent instability
  - This works well, but it is
    - Complex
    - Expensive
    - Hard to tune
    - Prone to instability
- DPO was introduced to
  - Achieve similar alignment results using a simpler and more stable objective

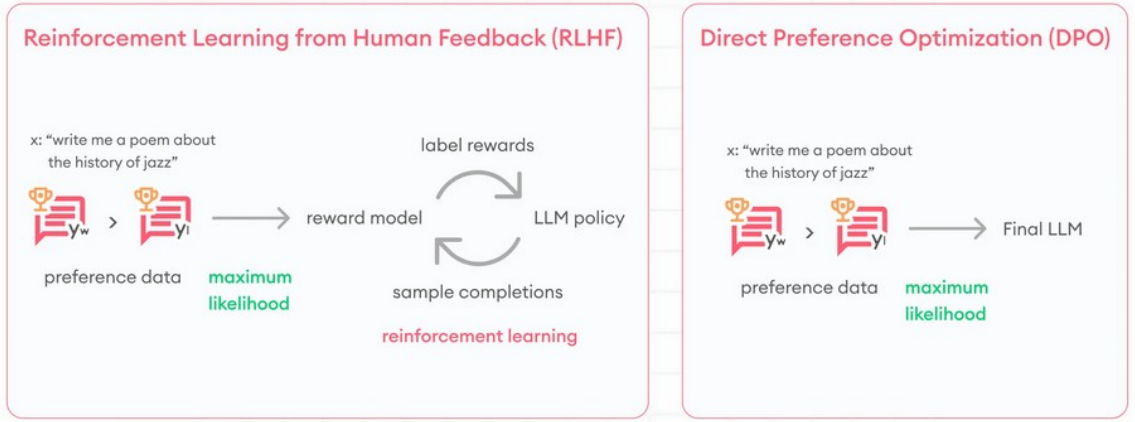


Image Credit: https://www.superannotate.com/blog/rlhf-for-llm

Direct Preference Optimization

- How DPO works
  - Each training example has
    - A prompt
    - Preferred responses
    - Rejected responses
  - The model is updated so that
    - The preferred response becomes more likely
    - The rejected response becomes less likely
  - And the model is constrained not to drift too far from the base model

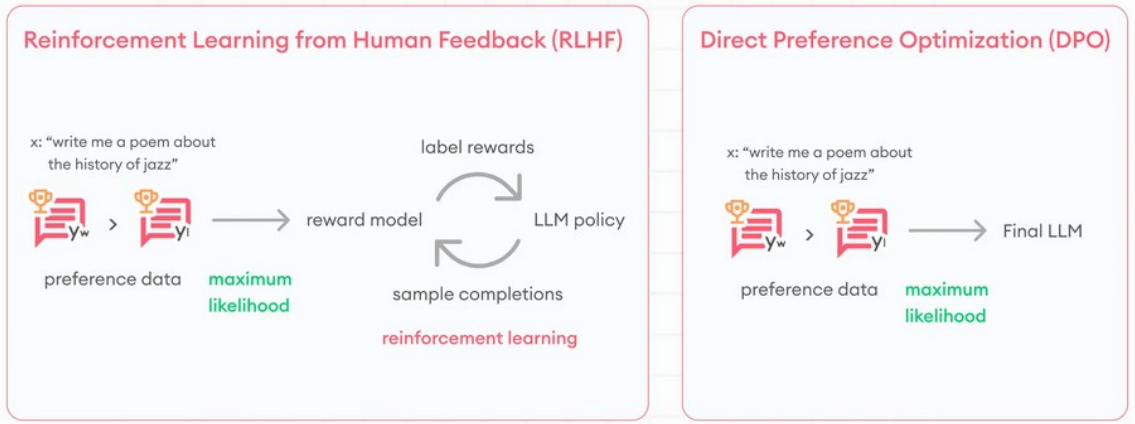


Image Credit: https://www.superannotate.com/blog/rlhf-for-llm

Direct Preference Optimization

- Why DPO works
  - Directly model preference between two outputs
  - Doesn’t need a separate reward function
- Relies on the fact that
  - Human preference comparisons already contain enough information
  - So that the model’s probabilities can be optimized directly
- Alignment is now:
  - A preference-based likelihood problem
  - Rather than a reinforcement learning problem

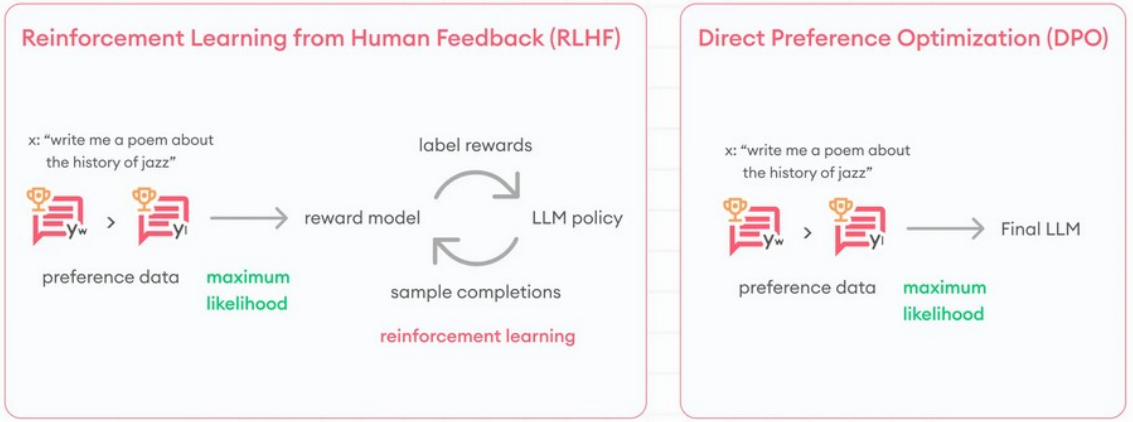


Image Credit: https://www.superannotate.com/blog/rlhf-for-llm

LLM Size

- The size of a LLM usually means
  - The number of parameters in the neural network
  - Examples
    - GPT-2 has 1.5 billion parameters
    - GPT-3 has 175 billion parameters
    - GPT-4 has 1.7 trillion parameters
- Parameters represent
  - The model's capacity to store patterns
  - The complexity of relationships it can represent
  - More parameters = more expressive power

	Parameters	Decoder layers	Context lenght	Hidden layer size
GPT-1	117 million	12	512	768
GPT-2	1.5 billion	48	1024	1600
GPT-3	175 billion	96	2048	12288
GPT-4	1.76 trillion	120	8000*	20k*

*Data subject to confirmation by OpenAI. Last updated: July 2023.

Image Credit: https://www.superannotate.com/blog/rlhf-for-llm

LLM Scaling

- Model performance improves predictably when scaling occurs across three dimensions
- Model size (parameters)
  - Larger models can represent more complex functions
- Training data
  - More tokenized input training data means better statistical estimation of language patterns
- Compute
  - More training steps and more GPUs/TPUs refine the model and produce more precise internal structures
- These relationships are called the scaling laws

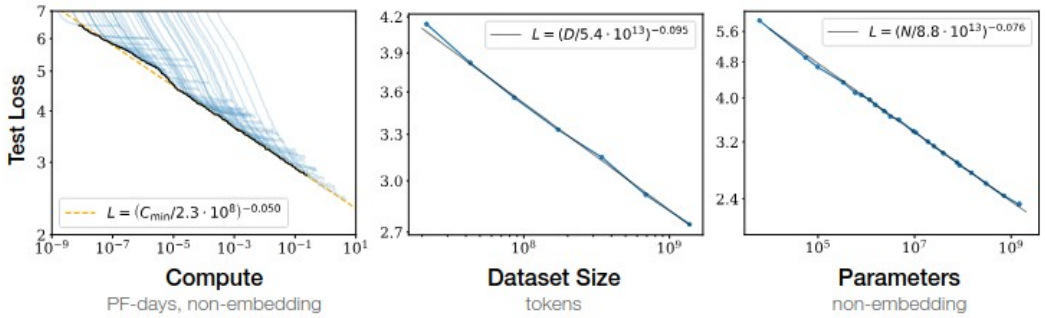


Figure 1 Language modeling performance improves smoothly as we increase the model size, datasetset size, and amount of compute used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

Image Credit: https://llmstudio.ru/blog/pretrain-llm-scaling-laws

LLM Scaling

- When models scale up they begin to exhibit
  - Better reasoning
  - Improved few-shot learning
  - More coherent long outputs
  - Better code generation
  - Stronger generalization
- Performance tends to improve smoothly as size increases

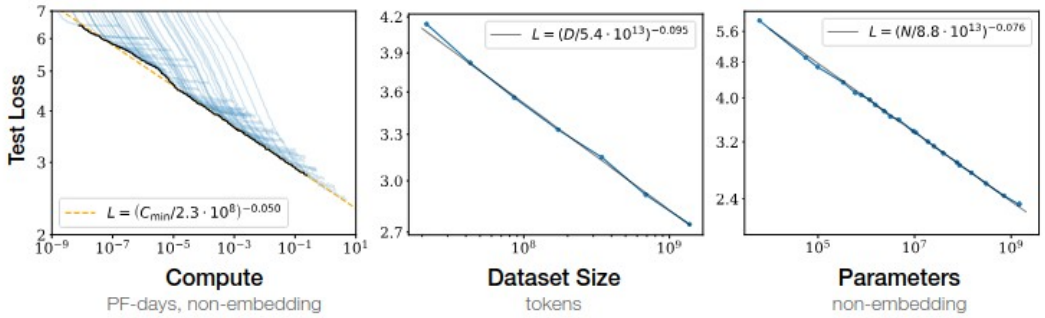


Figure 1 Language modeling performance improves smoothly as we increase the model size, datasetset size, and amount of compute[2] used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

Image Credit: https://llmstudio.ru/blog/pretrain-llm-scaling-laws

Emergent Behavior

- Emergent behavior means
  - A capability appears suddenly once the model reaches a sufficiently large scale
  - Examples observed in large models
    - In-context learning (learning from examples in the prompt)
    - Multi-step reasoning
    - Chain-of-thought reasoning
    - Arithmetic competence
    - Complex code generation
    - Tool-use planning behavior
- Smaller models may fail completely at these tasks
  - Larger models suddenly perform well

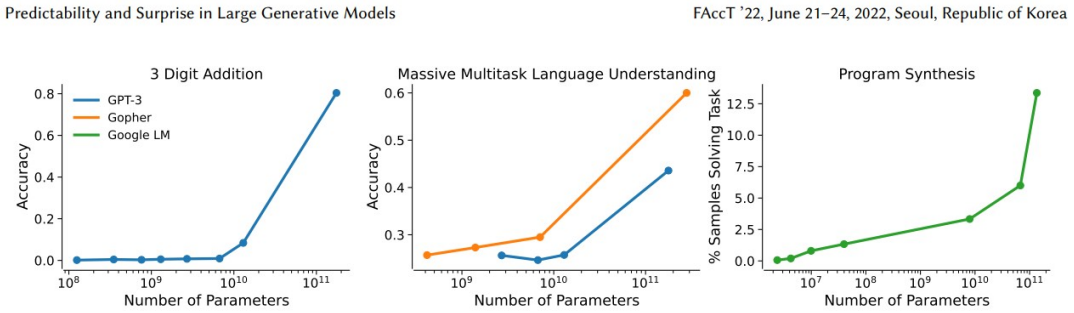


Fig. 2 Three examples of abrupt specific capability scaling described in Section 2.2, based on three different models: GPT-3 (blue), Gopher (orange), and a Google language model (green). **(Left)** 3-Digit addition with GPT-3 [11]. **(Middle)** Language understanding with GPT-3 and Gopher [62]. **(Right)** Program synthesis with Google language models [4].

Image Credit: https://www.christophgoetz.com/the-unpredictable-evolution-of-ai-challenges-in-testing-and-regulating-emergent-behaviors/

Emergent Behavior

- Emergent behavior are threshold effects
- Certain reasoning abilities require
  - Enough parameters
  - Enough training data
  - Enough depth
- Below a threshold, this capability is weak and unusable
- Above a threshold, this capability becomes usable

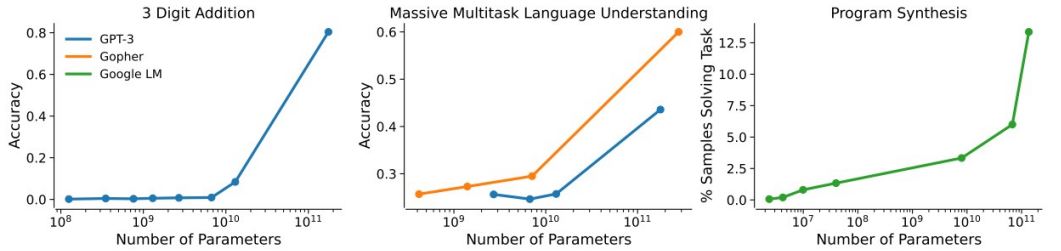


Fig. 2 Three examples of abrupt specific capability scaling described in Section 2.2, based on three different models: GPT-3 (blue), Gopher (orange), and a Google language model (green). **(Left)** 3-Digit addition with GPT-3 [11]. **(Middle)** Language understanding with GPT-3 and Gopher [62]. **(Right)** Program synthesis with Google language models [4].

Image Credit: https://www.christophgoetz.com/the-unpredictable-evolution-of-ai-challenges-in-testing-and-regulating-emergent-behaviors/

Emergent Behavior

- As models grow
  - Representations become richer
  - Abstractions become more layered
  - Internal reasoning chains become deeper
- Deep stacking and larger scale allows
  - Multi-step transformations
  - More complex internal “thought-like” structures

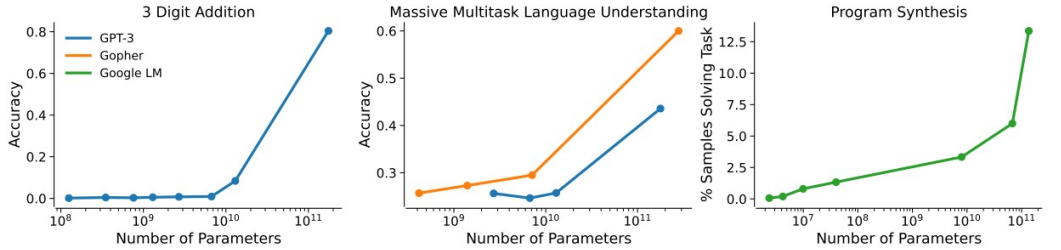


Fig. 2 Three examples of abrupt specific capability scaling described in Section 2.2, based on three different models: GPT-3 (blue), Gopher (orange), and a Google language model (green). **(Left)** 3-Digit addition with GPT-3 [11]. **(Middle)** Language understanding with GPT-3 and Gopher [62]. **(Right)** Program synthesis with Google language models [4].

Image Credit: https://www.christophgoetz.com/the-unpredictable-evolution-of-ai-challenges-in-testing-and-regulating-emergent-behaviors/

The Core Problem

- When a model generates code
  - It samples from a smoothed probability distribution
  - Rare events become even rarer
  - Common patterns become more dominant
- If a new model is trained on generated output
  - It learns the distorted smoothed distribution
  - Rare but important patterns disappear
  - Diversity shrinks
- After multiple generations
  - The distribution converges to a very small set of outputs
  - This is called model collapse

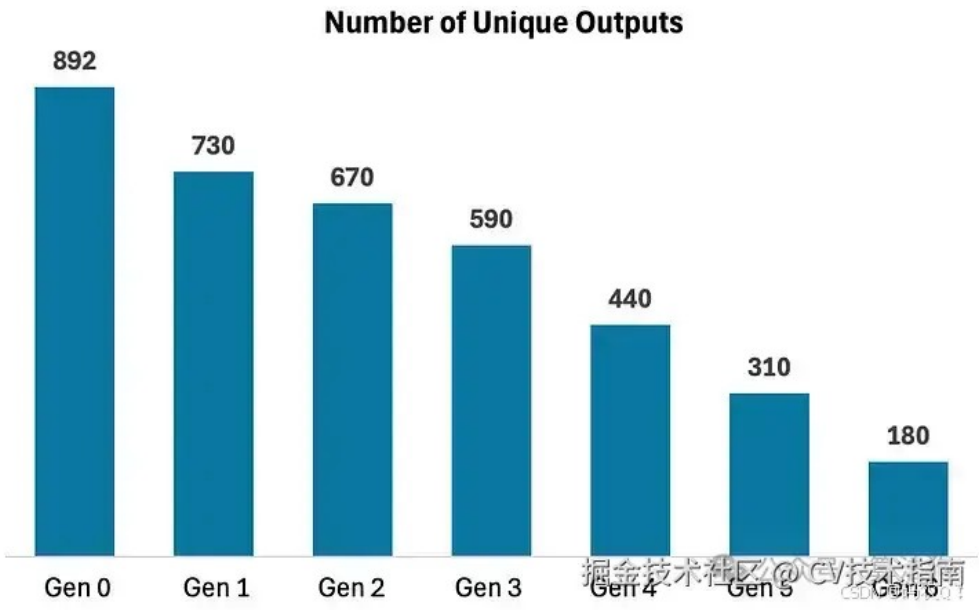


Image Credit:https://blog.csdn.net/Q2024107/article/details/146428423

The Core Problem

- The internet is increasingly filled with
  - AI-generated articles
  - AI-generated code
  - AI-generated forum posts
  - AI-generated marketing content
- Future models may scrape
  - AI code that was generated by earlier models
  - So training data becomes contaminated with LLM generated content
- Similar in concept to genetic anomalies introduce by inbreeding in a population

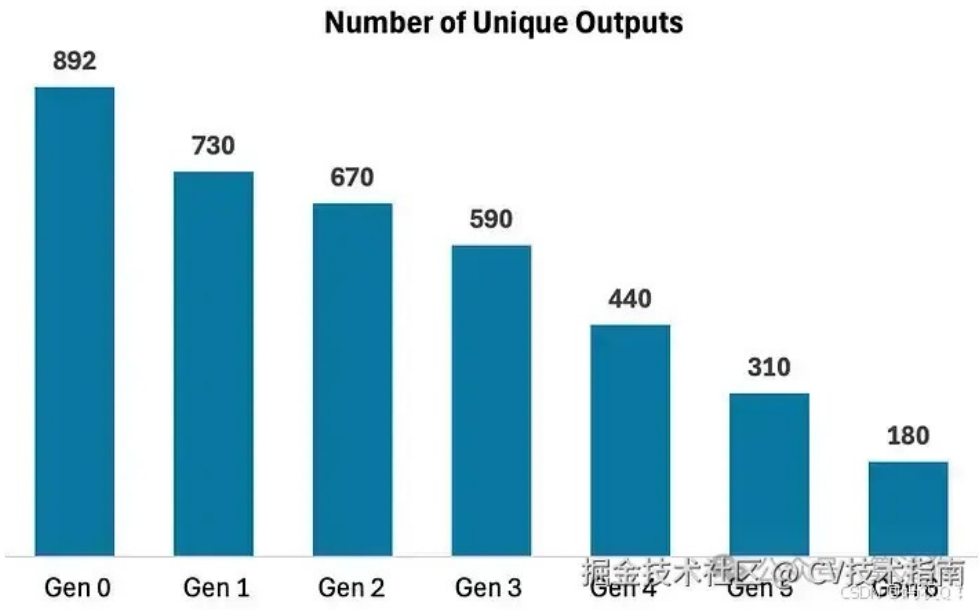


Image Credit:https://blog.csdn.net/Q2024107/article/details/146428423

Hallucinations

- A hallucination occurs when a model generates information that is false, fabricated, or unsupported
  - Examples
    - Inventing academic citations
    - Creating fake legal cases
    - Fabricating statistics
    - Misstating historical facts
    - Generating imaginary test cases
    - Writing code that compiles but does not execute correctly
- The model does not “know” it is hallucinating

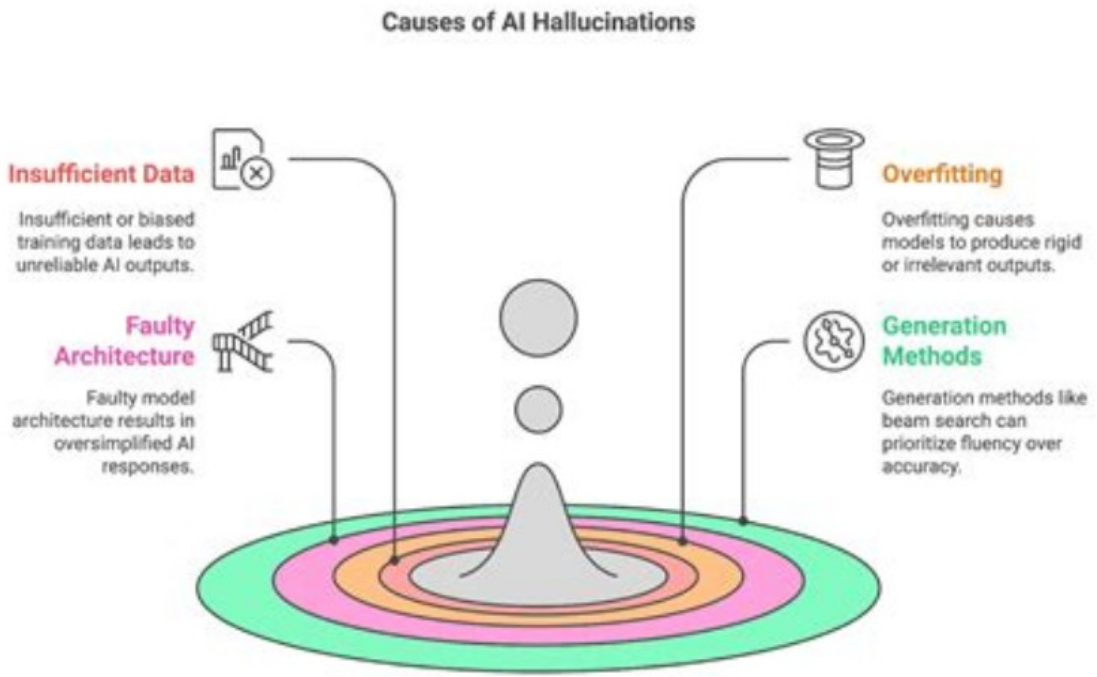


Image Credit: https://www.datacamp.com/blog/ai-hallucination

Hallucinations

- LLMs are trained to
- Predict likely next tokens
  - Not verify truth
  - They model probability, not reality
- If a plausible-sounding answer fits the language or code pattern
  - The model may produce it
  - Even if it is incorrect

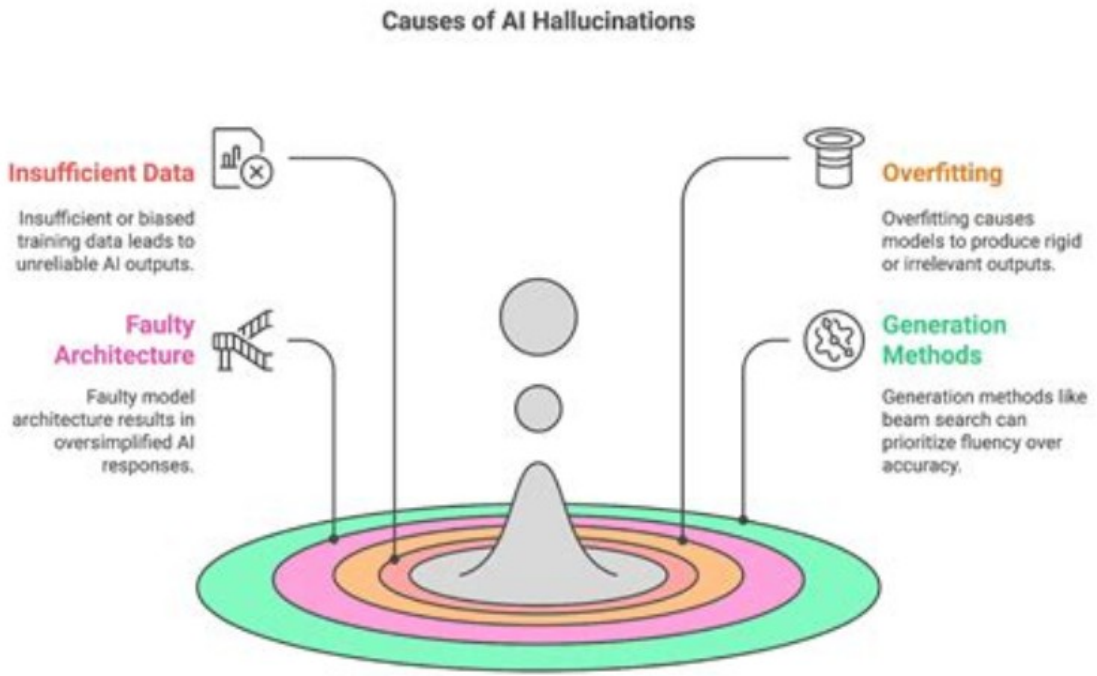


Image Credit: https://www.datacamp.com/blog/ai-hallucination

Hallucinations

- Why this is dangerous
  - High confidence can mislead users
  - Hard to detect without external validation
  - Especially risky in healthcare, law, finance
- Mitigation strategies include
  - Retrieval-Augmented Generation (RAG)
  - External verification
  - Confidence scoring
  - Human oversight

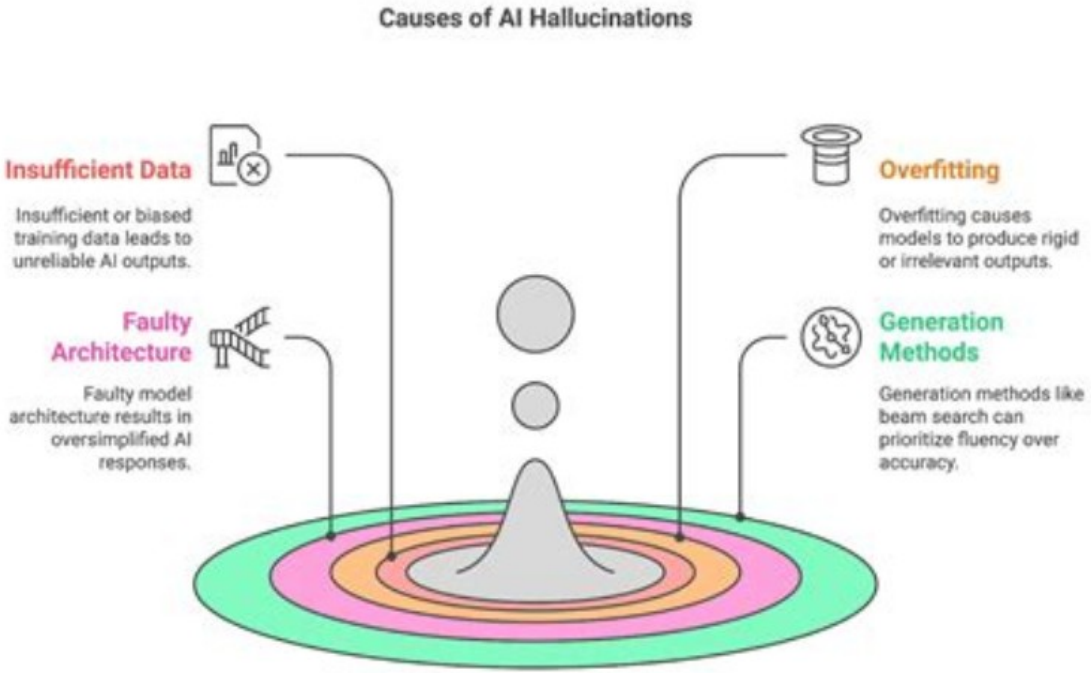


Image Credit: https://www.datacamp.com/blog/ai-hallucination

Bias and Toxicity

- Source of bias
  - LLMs are trained on large internet-scale datasets, which contain
    - Social biases
    - Stereotypes
    - Historical inequities
    - Toxic language
    - Badly written code
- Models may reproduce or amplify these patterns
  - The mostly likely code to be generated is “average” code
  - Training data includes badly written code as well as expertly written code

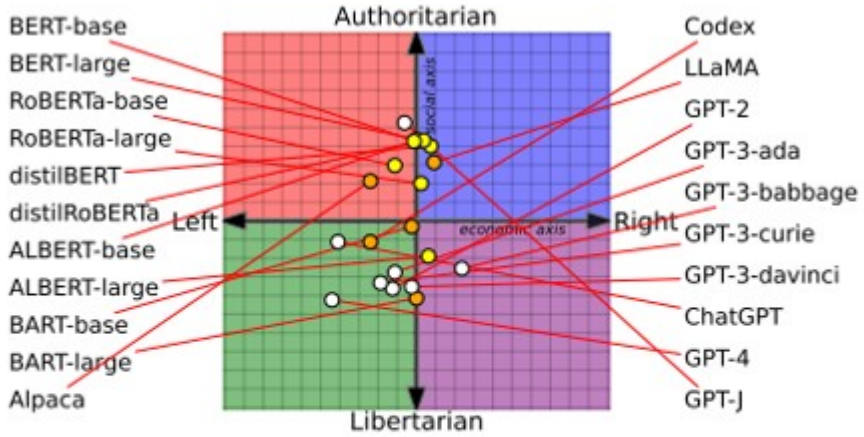


Figure 1: Measuring the political leaning of various pretrained LMs. BERT and its variants are more socially conservative compared to the GPT series. Node color denotes different model families.

Image Credit: https://glassboxmedicine.com/2023/11/28/bias-toxicity-and-jailbreaking-large-language-models-llms/

Data Leakage Risks

- LMs may unintentionally
  - Reveal sensitive training data
  - Reproduce memorized private information
  - Expose confidential user input
- Risk Scenarios
  - Personal data included in training corpus
  - Enterprise documents used in fine-tuning
  - Improper isolation between users
- Mitigation includes
  - Careful dataset curation
  - Differential privacy
  - Strict data governance
  - Isolation in enterprise deployments

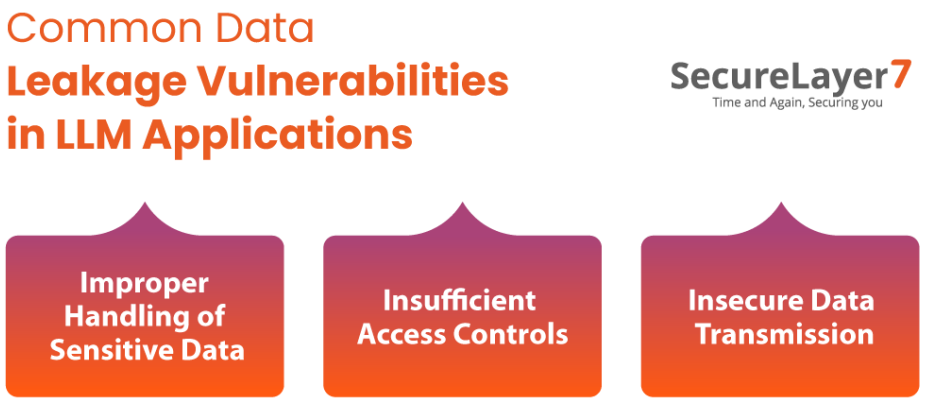


Image Credit: https://blog.securelayer7.net/owasp-top10-for-large-language-models/

Reliability Challenges

- LLMs can produce different answers to the same question because of
  - Probabilistic sampling
  - Sensitivity to phrasing
  - Context effects
- Non-Determinism
  - Even slight prompt changes can lead to
  - Different reasoning paths
  - Different conclusions
- This makes
  - Testing harder
  - Compliance auditing complex
  - Reproducibility challenging

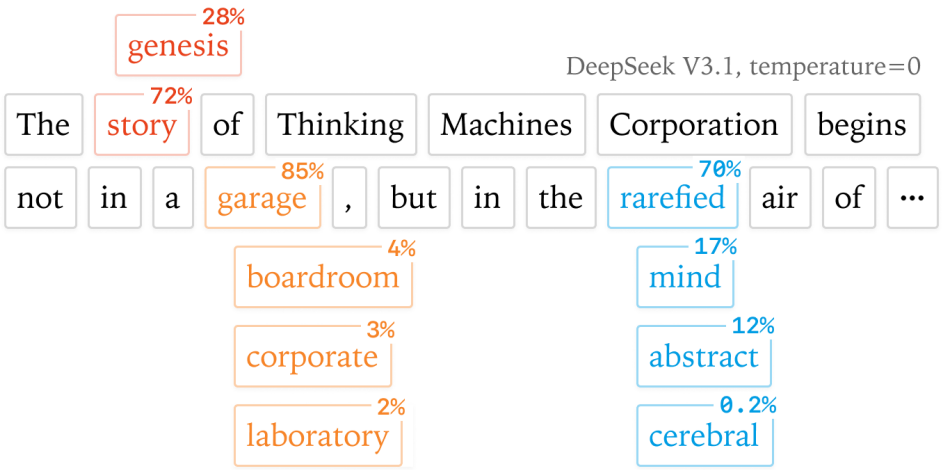


Image Credit: https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/

Lack of Ground Truth Awareness

- LLMs do not
  - Know when they are uncertain
  - Explicitly track factual grounding
  - Understand real-world consequences
- Reliability often requires
  - Guardrails
  - External tools
  - Structured pipelines

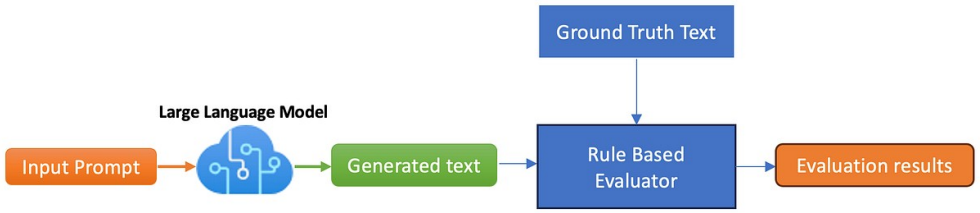
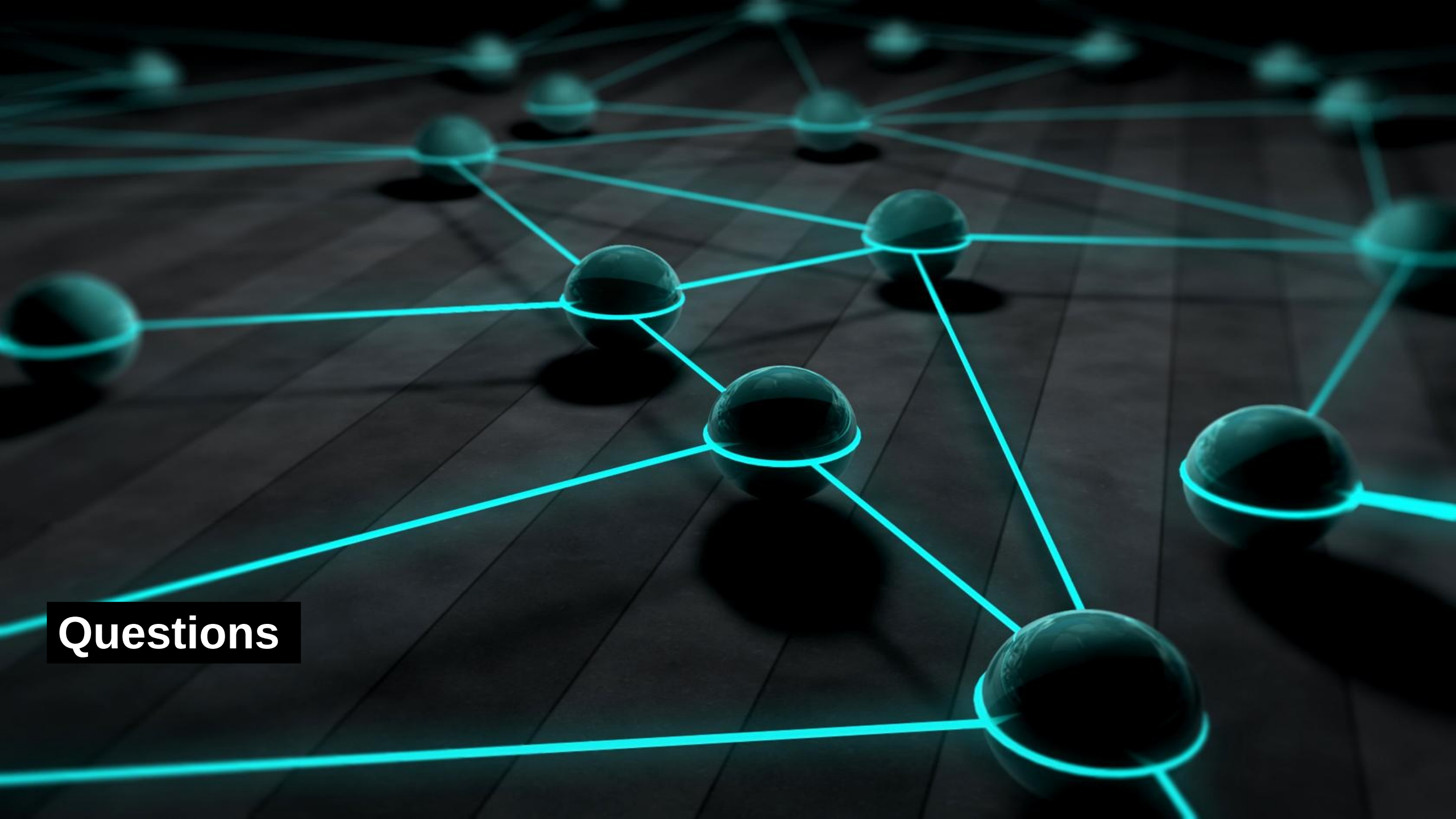


Image Credit: https://medium.com/@sathyavikram/part-2-beyond-the-buzzwords-rules-based-strategies-for-evaluating-llm-output-77fcd50be1b4


Questions